

# Instructor Handout 1: Where Does Your Data Live?

## Structure, Pointers, Dynamic Memory, and Simple Contracts

Dr. Auqib Hamid Lone

*Department of Computer Science & Engineering, GCET Ganderbal*

August 13, 2026

**Instructor Copy — Not for Student Distribution**

### 1.1 Before You Teach This Lecture

- **Wirth, *Algorithms and Data Structures*, §4.1, p.109** — the recursive-variability motivation (why a variable-sized structure cannot be given a fixed address at compile time); **§4.2, p.111** — the pointer / dynamic-allocation anchor itself. *Page-cited*; index built 2026-08-11. If you read one thing before class, read the §4.2 opening paragraph.
- **Sedgewick & Wayne, *Algorithms*, §1.2, p.64** — the ADT/data-type definition; §1.3, p.120 — bag/queue/stack as three collection types differing only in which object is removed next; §1.4, p.172 — the “how long will my program take? why does my program run out of memory?” framing this handout uses to motivate Week 2. *Page-cited*. All in Java — convert syntax to C per the notation registry, never present it as C.
- **Karumanchi, *Data Structures And Algorithms Made Easy*, Ch. 1, pp.18–19 (PDF)** — the two-part ADT definition (“declaration of data” + “declaration of operations”) and its commonly-used-ADTs list, used here to preview the Weeks 3–12 roadmap. *Page-cited*; index built 2026-08-11. **Not** a source for this lecture’s structured-programming or `malloc/calloc/realloc/free` content — indexing (Step 0.5) found zero coverage of either in this book. Reserve it for the ADT section only.
- **Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 1** and **Aho–Hopcroft–Ullman, *Data Structures and Algorithms*, Ch. 1** — *not page-indexed yet*; cite at chapter level only, for structured programming and the general ADT framing. Do not invent a page number for either.
- **Scope notice (updated from the prior deck):** this rebuilt, intro-tier deck now covers the *full* allocation family — `malloc`, `sizeof`, `calloc`, `realloc`, `free` — in one sitting. The earlier core-tier deck deferred `calloc/realloc` to a later week; that is no longer the plan. Budget real board time for both, not just `malloc`.
- **Plan the board examples in advance** (every section below, marked **Board:**). The stack trace, the heap trace, and the `calloc/realloc` worked traces are the ones to rehearse silently once before class — the numbers on the board must be right the first time, especially the `realloc` slot-by-slot trace, which is easy to fumble live.

## 1.2 Motivation and the Course Plan

This rebuilt deck opens with a full Act before touching memory at all. Its job is motivational and orienting, not technical — nothing here is examined directly, but it sets the frame the rest of the term (and this very lecture) leans on. Do not rush it to “get to the real content”; the real content is more legible once students know why they are learning it.

**Open with the four questions, on the board, before any answer.** Write, in order: (1) how do we organise a program’s *logic* before we organise its data; (2) where does a running C program keep its data; (3) how can we use memory that we request while the program runs; (4) how can we describe a data structure before implementing it. Compress the plan into one line and leave it on the board all lecture: *structure the logic, draw the memory, use one pointer, write one simple contract* — in that order, which is also the order of every section below.

**Board: three prediction boxes.** Draw three labelled boxes (“where,” “how,” “describe” — questions 2–4; question 1 gets answered by the very next Act) and invite one-word guesses from the class before revealing anything. Leave the boxes unfilled and return to them at the Week 1 summary. This costs two minutes and gives the whole lecture a spine students can feel closing.

**Teaching note:** The trap on the first frame is jumping straight to vocabulary. Spend real time making `int marks = 80`; concrete — it has a name, a value, *and* an address — before naming the stack or the heap. Use the house-number line: “an address is like a house number; it tells us where to look.” Students who skip this step later confuse a pointer’s *value* (an address) with the *thing at that address*, which is the single most persistent bug of the whole term.

**Board: a variable and its place.** Draw one box for `marks`, put 80 inside it, and write an invented address (say `0x7FFC`) above the box. Ask: “if we only knew the value, could we tell this box from its neighbour? What else do we need?” Answer: the address — exactly what a pointer will store, three sections from now.

**Motivation: correct is not always fast enough.** Use the student-record story: a loop that scans records is correct at 5 records and unusable at 10 million. The sentence to land, verbatim if you like: *the answer stays right; the arrangement of the data is the difference*. This single sentence justifies the entire course.

**Board: the record-scan trace (distinct instance 1).** Write “5 records — instant” and “10,000,000 records, one query per millisecond — unusable” on two lines, then draw an arrow between them labelled “same correct loop.” Ask: “what changed?” Not the loop — the arrangement.

**Board: “it felt fast” is not an argument (distinct instance 2).** Sedgewick’s §1.4 opener (p.172) poses “how long will my program take?” and “why does my program run out of memory?” as too vague to answer precisely without a model. Write both questions on the board and ask for a show of hands: “can anyone answer either question right now, for any program?” No one can — that is the hook for Week 2. Land the contrast explicitly: Week 1 supplies the raw material (memory layout, pointers); Week 2 supplies the model (counting operations,  $O/\Omega/\Theta$ ). This makes Week 2 feel like Week 1’s payoff rather than a new topic dropped in from nowhere.

**The course in one roadmap.** Walk the six clusters in order, spending no more than two minutes total: Weeks 1–2 foundations; 3–5 linear structures (arrays, stacks, queues, expression processing); 6–7 linked structures; 8–9 searching and sorting; 10 files and hashing; 11–12 trees, BSTs, AVL, graphs. State the recurring three-beat shape once, clearly: *state the contract, then implement it, then analyse what it costs*. Every week repeats it; students should be able to recite it back by Week 3.

**Teaching note:** Students commonly ask “why are we learning memory stuff in a data-structures course?” The roadmap is the honest answer: every later structure is a claim about

*where data lives and how it is reached*, so the memory model is not a prerequisite bolted on before the “real” course — it *is* the subject, applied one structure at a time. Come back to this line at the Week 1 summary; it lands harder the second time.

**The running thread: the expression processor.** Introduce the small program that reads `(a+b)*c` and must check its brackets, convert it, evaluate it, and look up its identifiers. Map each future week onto a deficiency it fixes: a stack checks the brackets (Weeks 3–4); a symbol table, first a list (Week 6) then a hash table (Week 10), stores the identifiers; an expression tree evaluates it (Week 11). State once, clearly, that this is one program learning new tricks each week, not twelve unrelated topics — it is the single best tool you have all term for making a new week feel motivated rather than arbitrary.

**What you already know.** The bridge slide: variables, arrays, functions, loops, `struct` — all from prior courses, none re-taught here. The one new thing this course adds is the question *what arrangement of data fits this job?*, answered every week with a contract, an implementation, and a cost.

## 1.3 Structured Programming: Disciplining the Logic Before the Data

**Read alongside:** standard treatment; see Horowitz & Sahni, Ch. 1 (chapter-level) [1].

This is a genuinely new Act relative to the prior core-tier deck, and it earns its place: before any data structure, this course wants students fluent in exactly three control patterns, because the discipline it teaches for organising *logic* is about to be reapplied, one level up, to *data*.

**Definition (Structured programming).** *The discipline of building a program from exactly three control patterns — sequence, selection, and iteration — instead of unrestricted jumps.*

State the three patterns plainly: **sequence** means doing one thing, then the next; **selection** means choosing exactly one branch, written in C as `if/else`; **iteration** means repeating work while a condition holds, written as `for` or `while`. Every C program students have ever written — however elaborate it looks — is built from nothing but these three shapes. This is worth saying explicitly: it is not a simplification for beginners, it is a theorem (the structured-program theorem) that this course states informally and uses operationally.

**Board: the larger function, build order.** Write the task first: “print the larger of two exam marks.” Then build the function top to bottom, narrating each line against a pattern as you write it:

1. Write the signature `int larger(int a, int b) {` and say: “the parameters arriving is *sequence* — two values, one after the other, before anything else happens.”
2. Write `if (a > b)` and say: “this is *selection* — exactly one of the two branches below executes.”
3. Write `return a;` then `return b;` and close the brace.
4. Ask: “which pattern is missing?” (Iteration — there is no loop, because comparing exactly two values needs only sequence and selection.)

**Example 1.3.1** (The larger function). 

```
int larger(int a, int b) { // sequence
    if (a > b)               // selection
        return a;
```

```

    return b;
}

```

*No loop appears anywhere in this function — comparing two values needs only sequence and selection. A loop appears the moment the task changes to comparing more than two marks, because then the same comparison must repeat once per additional value.*

**Board: distinct instance — three marks, not two.** Ask the class to extend larger to three marks *before* you write it, then build the answer live:

```

int largest3(int a, int b, int c) {
    int m = a;           // sequence
    if (b > m) m = b;     // selection
    if (c > m) m = c;     // selection
    return m;
}

```

The live-interaction hook: pose the question *before* revealing the code — “does this version need a loop?” Let the room split, then resolve it: still no, because the count of marks (three) is fixed and known at compile time; two `if` statements suffice. Push one step further with a second question: “what would force a loop?” Answer: an *unknown* count of marks, e.g. reading marks from an array of unspecified length — that needs a `for` loop over the array. This is the boundary the deck’s frame gestures at without stating it symbolically, and making it explicit here is worth the extra ninety seconds.

### 1.3.1 Why structure the program before structuring the data

The reason this course opens with a program-organization idea rather than a data-organization idea: the same discipline is about to be reapplied to data, one level up. A function is a **named, reusable block** — exactly the idea this course will apply to data: name a contract, then implement it. Top-down design proceeds the same way for programs as it will for data structures: state the goal, break it into smaller pieces, then fill each piece in. There is also a practical payoff specific to this course’s subject matter, and it is worth stating out loud: well-structured code is what makes pointer and memory bugs *findable* — a tangled program hides them, because a bug in an unstructured jumble of jumps could be almost anywhere, while a bug in a disciplined sequence of named functions is localized to one function’s sequence/selection/iteration logic.

**Teaching note:** The recurring student misconception here is thinking this Act is a review lecture they can tune out of because “I already know `if/else` and `for` loops.” Redirect that: the content is familiar, the *claim* is not — that this exact discipline (name the contract, then implement it) is the organizing principle for the rest of the term, applied to data instead of control flow. Say this explicitly, because the payoff (Section 1.7 below) is six sections away and the connection is easy to lose.

**Live question:** “a function is a named, reusable block of *logic*. What will the data equivalent turn out to be called?” Do not answer yet — let the word “contract” or “ADT” surface naturally when Section 1.7 arrives; planting the question here makes that section’s payoff land as a recognition, not a new idea.

## 1.4 The Memory of a Running Program

**Read alongside:** Wirth, *Algorithms and Data Structures*, §4.1, p.109 (motivation) and §4.2, p.111 (the pointer/dynamic-allocation argument); Horowitz & Sahni, Ch. 1 (chapter-level).

State the two-part geography and keep the orientation fixed for the entire term: the **stack** (short-lived local variables) above, the **heap** (memory requested at run time) below. Figure 1.1 is the simple course picture — every later memory diagram in the course (Weeks 6–7, 11) reuses this exact orientation, so getting it right here matters more than it looks.

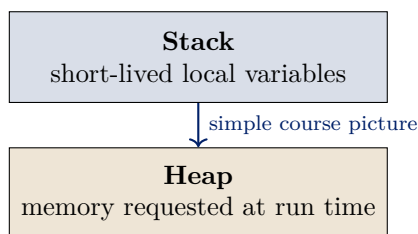


Figure 1.1: The two places a running C program keeps data. Orientation (stack above, heap below) is fixed for the whole course.

**Board:** draw the two boxes first. Stack on top, heap on the bottom, in that order, before saying anything about growth direction — growth direction is the next move (below) and introducing it here dilutes the simple two-region picture students need to keep for the rest of the term. Label them only “short-lived locals” and “run-time requests” for now.

### 1.4.1 The stack grows: a three-step story

A variable declared inside a function is created when the function starts, is available while that function is running, and disappears when the function returns — the program manages this basic lifetime *automatically*, with no explicit allocation and no explicit cleanup. To make “automatic” concrete rather than a word students nod at, trace what happens to the stack as `main` calls `add(x, y)`, which computes `int s = x + y`; and returns it.

**Example 1.4.1** (Stack growth, Step 1: `main` starts). *When the program starts, the system creates one **frame** on the stack for `main`, holding `int a = 5`; . At this instant only this one frame exists (Figure 1.2); the space below is free, waiting for the next function’s frame.*

**Example 1.4.2** (Stack growth, Step 2: `add` is called). *The call `add(x=5, y=3)` pushes a second frame just below `main`’s, holding the parameters and `int s = x + y; = 8` (Figure 1.3). Every new function call pushes a new frame just below the current one, so each new frame sits at a **lower** address than the frame above it.*

**Example 1.4.3** (Stack growth, Step 3: `add` returns). *When `add` returns, its frame is popped — it no longer exists, and its memory is available again (Figure 1.4). The answer 8 is copied back into `main`’s frame as `int s = 8`; . The stack has returned to its Step 1 shape.*

**Board:** live stack trace (interactive, distinct instance 1). Draw the empty board frame ahead of time. Ask one student to “call” `add` by walking to the board and drawing a new frame below `main`’s, labelling its contents; ask a second student to “return” by erasing that frame and writing `s = 8` into `main`’s. Ask mid-way: “where does the next frame go — higher or lower address?”

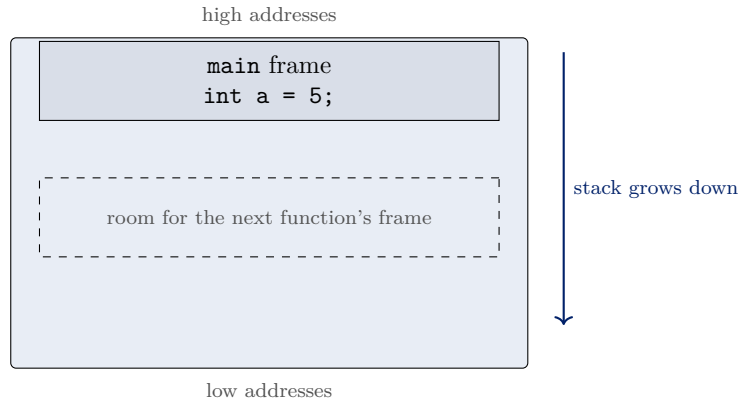


Figure 1.2: Stack growth, Step 1: `main` starts, its frame is created holding `int a = 5;`. The stack grows *downward*.

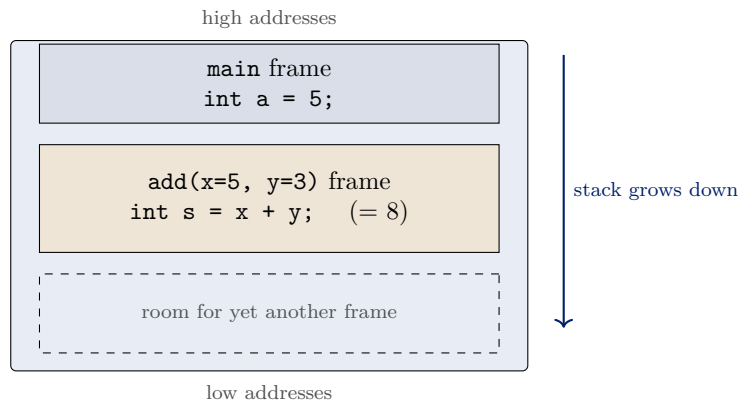


Figure 1.3: Stack growth, Step 2: the call to `add` pushes a second frame at a lower address than `main`'s.

(Lower — the stack grows down.) Two volunteers, three steps, the whole class watches the stack grow and shrink.

**Board: distinct instance 2 (prediction trap).** Pose a different pair: `main` holds `int n = 3;` and calls `doubleIt(n)` whose local is `int d = 2*n;`. Ask the class to draw the two-frame picture *before* you do, including which region is the higher address. Reveal after a show of hands. This is a genuinely different pair of functions from the deck's `main/add` example, so it exercises the same skill without being a restate.

**Teaching note:** The recurring error here is treating the stack as “a magic box that grows.” The fix is the low-address/high-address labels drawn every time — *directions* (heap up, stack down) get fixed once and reused for 12 weeks; vary them once and you silently break a later diagram (Weeks 6–7, 11 all assume this orientation). Also be explicit that *automatic* means “the program manages it with zero instructions from you,” not “I don’t have to think about it” — students conflate the two, and the conflation is exactly wrong once pointers into stack frames enter the picture (a dangling-pointer preview worth flagging even though the deck saves the term for the heap).

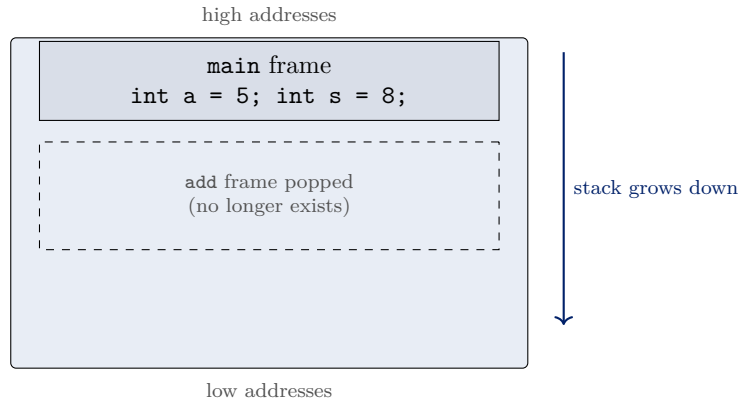


Figure 1.4: Stack growth, Step 3: `add` returns, its frame is popped, 8 is copied back into `main`’s frame.

### 1.4.2 Why the stack is not enough

State the requirement the stack cannot meet: the user enters the record count only after the program runs, and we may need 3 records today, 300 tomorrow. The stack decides frame sizes at compile time and discards locals on return — it fails on *size* and on *lifetime* simultaneously.

**Board: the guessing trap.** Write `int a[1000000];` and ask: “when is this wasteful, and when does it still fail?” (Wasteful when the actual count is small; fails outright the moment the real count exceeds a million — guessing rescues nothing.) This reuses Wirth’s p.111 argument without the formalism: a variable-sized structure “cannot be assigned a fixed amount of storage,” so allocation must happen *when the data comes into existence* [4], not at compile time.

**Board: the live prediction (distinct instance).** Put the two declarations on the board:

```
int score = 80;   vs.   int *p = malloc(sizeof(int));
```

Ask: “*which variable can outlive the function that created it?*” Let votes land before revealing. Answer: the *block*, not the variable that names it — the pointer is only the handle. Do not reveal `malloc` detail yet; you have told them the call exists, and that gap is the hook into the next two sections.

## 1.5 Pointers: The Handle on Memory

**Read alongside:** Wirth, §4.2, p.111 — the dynamic-allocation argument and the address-as-handle picture.

**Definition (Pointer).** *A variable that stores an address. Its type records what kind of object resides at that address* [4].

The contrast to draw explicitly: an ordinary variable stores a value such as 80; a pointer stores a *place*. `int *p;` means “`p` can point to an `int`.” Every pointer is a handle, never the data itself — this is the single sentence students most need internalized before Section 1.6 makes it operational.

**Board: box-and-arrow build order (distinct instance 1).** Draw the two boxes first, empty; write 80 in the right-hand box; only then draw the arrow, and label it “points to” last. Ask before drawing the arrow: “*what does `p` contain — the 80, or the address?*” (The address.) The order matters pedagogically: drawing the arrow last mirrors the fact that the pointer’s whole job *is* the arrow, not the boxes.



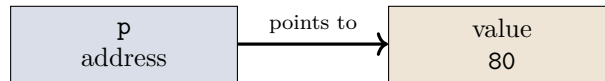


Figure 1.5: A pointer picture: `p` holds an address; the value 80 lives at that address.

### 1.5.1 Pointers reach structures, and stop cleanly

Introduce the two-field node, the building block for most of the term:

```
struct node {
    int data;
    struct node *next;
};
```

`data` stores the value; `next` stores the address of *another* node, or `NULL` if there is none. Establish the arrow field-access now, since the deck uses it from the first `malloc` example onward: `p->data` reads the `data` field of the node `p` points at.

**Definition (NULL).** *The address value meaning “points at nothing.” Reading through it is undefined behaviour — in practice, a crash.*

**Board: node-picture drill (distinct instance 2, interactive).** Draw three boxes — `head` at the left, then node 1 and node 2, each split into `data/next`. Ask the class to draw the arrows so `head` points at node 1, node 1’s `next` points at node 2, and node 2’s `next` is written `NULL`. Then erase the middle arrow and ask: “*what happens to node 2 now?*” Answer: reachable no longer — a preview of how a leak/dangling situation begins, even though the vocabulary is not introduced until Section 1.6.6.

**Board: distinct instance — extend the chain.** Ask the class to add a third node and mark where the chain terminates, then give the one-line meaning of `struct node *head = NULL;` (“the list is empty”). Two arrows and one `NULL` is a cheap repeat that fixes the convention (`head` as the list handle) before Week 6 needs it for real.

**Teaching note:** Students arriving from an earlier course will write `(*p).data` and resist `p->data` as “sugar.” Hold the line: the course standard is `->` everywhere, per the notation registry, and `(*p).data` — while correct C — is noise this course does not use. Expect the “is a pointer just a number?” question too: yes, an address is a number, but the pointer’s *type* records what kind of object lives there, which is exactly why `int *` and `struct node *` are different types even though both “just hold an address.”

**Live question:** “if `p` points to a node and we set `p = NULL`, what happened to the node?” (Nothing — the node is untouched; only the pointer’s value changed. This is the seed of the memory-leak idea, two sections early.)

## 1.6 One Heap Block: Allocate, Use, Return

The whole memory discipline of the course is three moves, applied to one block at a time: **request memory, use it, return it**. This section works through C’s complete heap-allocation interface in the order a program actually needs it — `malloc` and `sizeof` first, then the two allocation variants `calloc` and `realloc`, then the safe-node pattern, then the heap-growth picture, then `free` and the two beginner mistakes. Budget real time for all five calls; this is the largest section of the lecture and the one where board rehearsal pays off most.



### 1.6.1 Requesting one integer: malloc and sizeof

**Read alongside:** Wirth, §4.2, p.111.

State the pattern before explaining it:

```
int *p = malloc(sizeof(int));
if (p == NULL) return 1;
*p = 80;
```

Three facts to narrate in order: `malloc(n)` requests *n bytes* from the heap and returns their address, or `NULL` if the request fails; `sizeof(T)` is the compile-time size in bytes of type *T*, and it is what tells `malloc` how many bytes an `int` actually needs — `malloc` never knows this on its own; and `p` stores the returned address, not the data. Emphasise explicitly: never count bytes by hand.

**Board: same pattern, distinct type (instance 1).** Walk the identical three lines for `double` instead of `int`: `double *d = malloc(sizeof(double)); if (d == NULL) return 1; *d = 3.5;`. Ask: “*what changed between the `int` and `double` versions?*” Answer: only the type named in the declaration and inside `sizeof` — the shape is identical. This “one pattern, many types” payoff is worth making explicit; it is the reason `sizeof` exists rather than a hand-counted constant.

**Board: live failure trace (instance 2).** Ask: “*which line would fail if `p` is `NULL`?*” (Line three, the dereference `*p = 80;`.) If `malloc` returned `NULL`, an unguarded line three reads/writes through nothing — a crash. The `if` guard turns line two into a controlled exit. Correct the habit live: never let a line touch a pointer that *may* be `NULL` before it has been checked.

**Teaching note:** The most common lab mistake this week is dropping `if (p == NULL)` “because it never happens.” The trace above is the fix: the dereference is unprotected, so an out-of-memory condition silently becomes a crash at a random later line, far from the actual bug. Also do not let students write `malloc(sizeof(8))` — the argument to `sizeof` is a *type* or an expression whose type is inferred, not a numeric literal that happens to look like a size.

### 1.6.2 Zero-initialized memory: calloc

**Definition (calloc).** `calloc(n, size)` allocates space for *n* items of *size* bytes each, and sets every byte of that space to 0.

```
int *counts = calloc(5, sizeof(int));
if (counts == NULL) return 1;
```

The request above asks for room for five integers, exactly as `malloc(5 * sizeof(int))` would, but with one added guarantee: **calloc zero-initializes; malloc does not.** That guarantee is the entire difference — same style of request, same `NULL` check, different starting content.

Figure 1.6 draws the contrast directly: two independent four-byte requests side by side, with no line connecting them — they are unrelated blocks, shown together purely for comparison.

**Board: walk the diagram, build order (instance 1).** Draw the left box first, unlabelled, and ask students to guess its contents before revealing the four `?` marks — *unspecified* means the bytes could be anything left over from a previous use of this memory, including garbage that happens to look valid. Only then draw the right box with its four zeros, and underline: same size request, only the starting content differs.

**Board: distinct instance 2.** Trace `struct point *pts = calloc(3, sizeof(struct point));` where `struct point` has two `int` fields `x` and `y`.

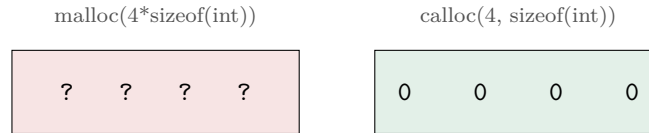


Figure 1.6: `malloc` vs. `calloc`: `malloc` hands back four unspecified bytes (reading before writing is undefined behaviour); `calloc` hands back four bytes already zeroed.

Ask: “what are `pts[0].x` and `pts[1].y` immediately after this call, before any assignment?”  
 Answer: both 0 — `calloc` zeroes every byte of the whole block, not just the first element, so every field of every struct in the array starts at zero. This generalizes the two-int example to the struct case students will actually use from Week 6 onward.

**Teaching note:** Students routinely assume `malloc` also zero-initializes, usually because a test run “happened to” print zeros (freshly-mapped OS memory is often zero, which is an implementation accident, not a guarantee). The fix is insisting on the vocabulary distinction every time: `malloc`’s bytes are *unspecified*, not *zero* — “it worked when I tested it” is not evidence of defined behaviour. `calloc` pays a small extra cost (writing the zeros) for the guarantee; that trade-off is worth stating explicitly so `calloc` does not look like a strictly-better free upgrade.

**Live question:** “if you `malloc` an array and then read element 2 before writing to it, what can you say about the value you get back?” (Nothing reliable — it is unspecified, and reading it before writing is undefined behaviour in the strict sense, not just “probably garbage.”)

### 1.6.3 Resizing a block: `realloc`

**Definition (`realloc`).** *`realloc(p, newsize)` resizes the block `p` points to, preserving its existing content up to the smaller of the old and new sizes. The block **may move** — always keep the returned pointer.*

```
int *bigger = realloc(p, 10 * sizeof(int));
if (bigger == NULL) return 1; // p is still valid here
p = bigger;
```

Two details in this idiom matter more than they look, and both are worth narrating explicitly rather than assuming students absorb them from reading the code: first, the check is against `bigger`, not `p` — if `realloc` fails it returns `NULL` but leaves the *original* block at `p` untouched, so overwriting `p` with a possibly-`NULL` result before checking would silently lose access to still-valid memory; second, `p` is reassigned to `bigger` only *after* the check succeeds.

**Example 1.6.1** (Growing an array with `realloc`). *Start with `p = malloc(3 * sizeof(int))` holding 10 20 30. Trace, one statement at a time, growing to five slots:*

1. `p = realloc(p, 5 * sizeof(int));` — requests a block of five integers, built from the existing block at `p`.
2. The first three ints (10, 20, 30) are **copied**, in order, by `realloc` itself — the caller does not write this copy.
3. The two new slots (index 3 and 4) are **unspecified** — `realloc` does not zero new space; only `calloc` zeros space.

If `realloc` returns `NULL`, the original block at `p` is untouched and still valid — never overwrite `p` with the raw result before checking it, exactly as in the idiom above.

**Board: the growth trace, build order (instance 1).** Draw the original three-slot block first: 10 20 30. Then draw a wider five-slot outline *next to it* (not overwriting), and narrate the copy happening slot-by-slot: “`realloc` copies 10 to slot 0, 20 to slot 1, 30 to slot 2 — then leaves slots 3 and 4 blank.” Only after the copy is drawn, mark slots 3–4 with ? and ask: “can we assume these are zero?” (No — only `calloc` guarantees that.)

**Board: distinct instance 2 — `calloc` then `realloc` together.** This is the combination students will actually hit in code, so it is worth a full live trace: `p = calloc(3, sizeof(int));` then `q = realloc(p, 6 * sizeof(int));`. Ask the class to predict, before you draw: “after both lines, what is in slots 0–2 of `q`, and what is in slots 3–5?” Build the answer on the board: slots 0–2 were zeroed by `calloc` and then *preserved* by `realloc`’s copy, so they remain 0; slots 3–5 are the newly added space, which `realloc` never zeroes, so they are **unspecified** exactly like a fresh `malloc`’s bytes. The one-line summary worth writing on the board: “*realloc preserves old content; it does not extend calloc’s zero guarantee into new space.*”

**Teaching note:** The two live-coding bugs to watch for: (1) writing `p = realloc(p, ...)` directly instead of assigning to a temporary first — if this fails, `p` is now `NULL` and the original block’s address is lost forever (a guaranteed leak, not just a missed error message); (2) assuming the two new slots in a grown block are zero because “the old slots were zero” (from a prior `calloc`) — students generalize `calloc`’s guarantee to `realloc` because the two calls feel like siblings. They are not: only `calloc` zeroes, and only at the moment of that specific call.

**Live question:** “if `realloc(p, newsize)` returns `NULL`, is the block at `p` still usable?” (Yes — explicitly, per the definition, the original block is untouched on failure. This is the fact the two-temporary idiom exists to protect.)

#### 1.6.4 Allocate a complete node: the safer engineering pattern

```
struct node *p = malloc(sizeof(struct node));
if (p == NULL) return 1;
p->data = 42;
p->next = NULL;
```

Four properties to narrate, in the order the code enforces them: `sizeof(struct node)` asks for exactly one node’s bytes, never a hand-counted 8 or 12 that breaks the moment a field is added or the code moves to another machine; the result is checked against `NULL` before anything touches the block; `p->data = 42;` sets the value field; `p->next = NULL;` makes the node’s *end* explicit. Initialising every field makes the node’s state predictable — an uninitialised field holds garbage, the same unspecified-content hazard from Section 1.6.2.

**Board: audit-the-node checklist (instance 1, interactive code review).** Write the four checks on the board and have the class *vet a snippet you show*, not one you narrate:

1. Was the allocation checked against `NULL`?
2. Was every field initialised before the node was used?
3. Is there exactly one clear owner responsible for `free(p)`?
4. Can the program reach the node later, or has its pointer been lost?

Show a deliberately broken variant — omit the NULL check and `p->next = NULL;` — and have students find both violations before you fix it live.

**Board: trace-before-run challenge (instance 2).** Put `int *p = malloc(sizeof(int)); *p = 80;` on the board and ask: “*what is true after these lines?*” Build the answer as three separate facts, not one sentence: `p` contains an address; the heap block at that address contains 80; the name `p` itself is *not* the heap block — they are two different objects in two different regions. Reuse Figure 1.7 as the shared reference picture, and pose the challenge live: “*which line would fail if `p == NULL`?*” (The second line, the dereference.)

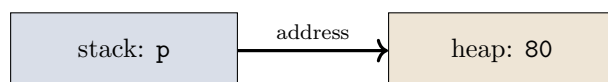


Figure 1.7: What happened: `p` lives on the stack and holds the address of a heap block containing 80.

### 1.6.5 How the heap grows: a three-step story

The converse of the stack trace: the heap grows *upward*, toward higher addresses. Three steps, mirroring the stack’s three-step story exactly, so this is a good moment to have students notice the parallel structure rather than treat it as new material.

**Example 1.6.2** (Heap growth, Step 1: one `malloc` for three integers). *One `malloc` request with room for three integers grows the used heap upward, placing a block holding 10, 20, 30 (Figure 1.8). The block survives after the allocating function returns — that is the heap’s entire point, the direct payoff of Section 1.4’s “why the stack is not enough” argument.*

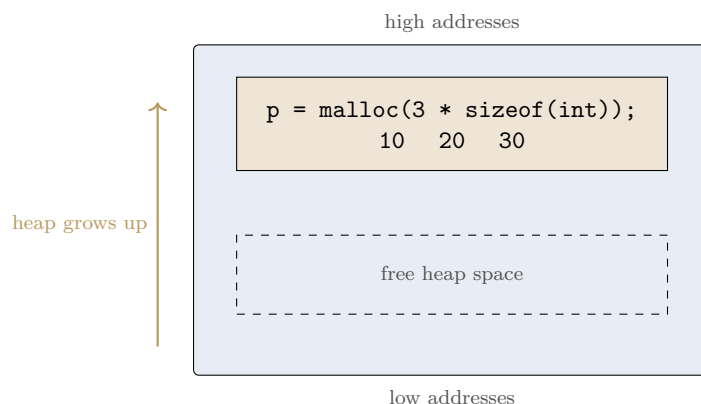


Figure 1.8: Heap growth, Step 1: `malloc(3 * sizeof(int))` places one block holding 10, 20, 30.

**Example 1.6.3** (Heap growth, Step 2: a second block). *The second request places `q`’s block at a **higher** address than `p`’s (Figure 1.9). Blocks are independent: each keeps its own data.*

**Example 1.6.4** (Heap growth, Step 3: `free` returns a block). *`free(p)` gives `p`’s block back to the system (Figure 1.10); `q`’s block is untouched, and the used heap can shrink again.*

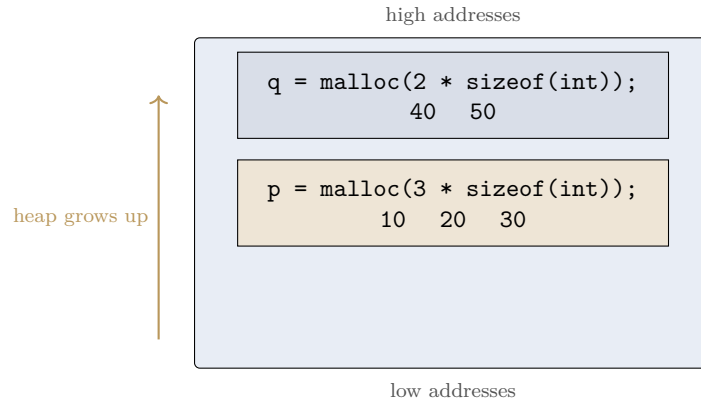


Figure 1.9: Heap growth, Step 2: a second `malloc` places `q`’s block at a higher address; the two blocks are independent.

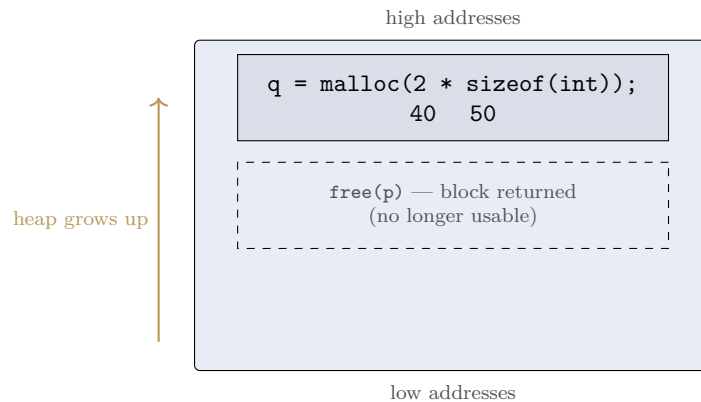


Figure 1.10: Heap growth, Step 3: `free(p)` returns `p`’s block; `q`’s block is untouched.

**Board: heap growth trace (instance 1, interactive, three volunteers).** Draw the empty heap column. Volunteer 1 “calls” `malloc(3 * sizeof(int))` and draws the block holding 10 20 30 near the bottom; volunteer 2 “calls” `malloc(2 * sizeof(int))` for `q` and draws its block *above* the first; volunteer 3 “calls” `free(p)` and shades the bottom region “free again.” The class watches the used heap grow up, then shrink from the bottom.

**Teaching note:** The single most instructive contrast in the lecture is running the stack trace and the heap trace back to back and asking: “*why opposite directions?*” Answer, if time permits (the deck says no more than this in Week 1): so the two regions can grow toward each other and share one contiguous free space without colliding — the real layout of a process’s address space. If you have covered both traces already, this is a two-sentence payoff; do not build a whole new example around it.

**Board: distinct instance 2 (prediction).** Cover Steps 2–3 and ask: “*after `free(q)`, where does a new `malloc` of three integers land?*” Let guesses land, then reveal: exact placement is the allocator’s choice; what matters is that freed space is *reusable*, not its guessed address. This inoculates students against the common bug of hard-coding or reasoning about specific addresses.

### 1.6.6 Returning the memory, and two beginner mistakes

The return idiom is two lines that travel together, always:

```
free(p);  
p = NULL;
```

`free(p)` returns the heap block to the system; `p = NULL` makes the old pointer visibly empty so it cannot be mistakenly used as though the block still existed. This is the deck’s summary line, and it is worth writing on the board verbatim: **request memory, use it, return it**. Every `malloc/calloc/realloc` block needs exactly one matching `free` — the “one” matters as much as the “matching.”

The two mistakes that violate this discipline:

- **Memory leak**: forget to call `free`; the block stays reserved forever, and if the only pointer to it is overwritten, nothing can ever reach it again. Nothing crashes — the program quietly grows until it runs out of memory.
- **Dangling pointer**: use a pointer after its block was freed. Reads return garbage; writes corrupt whatever now occupies that space.

**Board: the two mistakes side by side (instance 1, interactive clinical case).** Draw the heap with one live block, then:

1. Sketch the **leak**: erase the pointer variable’s box while leaving the heap block drawn — ask “*who can still reach this block?*” (No one; it is unreachable but still reserved.)
2. Sketch the **dangling pointer**: shade the freed block “free,” but keep the pointer’s arrow sharp and pointing at it — ask “*what does the arrow now point at?*” (Nothing usable; the block may already be handed to someone else.)

The live habit to land: the owner is whoever frees. Ask a student to restate who owns the node from Section 1.6.4’s worked example.

**Board: distinct instance 2 — the leak in a loop.** Show a program that allocates one node per loop iteration and never frees any of them. Trace the heap growing block after block with no volunteer ever shading anything “free.” Ask: “*what eventually happens to this program?*” (It runs out of memory and dies — nothing crashes on the first iteration, or the tenth, which is exactly why leaks are dangerous: the failure is delayed and looks unrelated to its cause.)

**Teaching note:** Expect students to `free(p)` without setting `p = NULL`, then dereference the stale pointer in a later function and blame the compiler. Teach `free(p); p = NULL;` as *one gesture*, not two independent lines — say it as a single breath when narrating code. Also expect the “I freed it but nothing bad happened” shrug after a lab exercise: the bug is silent today and fatal on tomorrow’s larger run, or on a different machine’s allocator. That silence is the entire reason this discipline is taught as a habit rather than left to intuition.

**Live question:** “between a memory leak and a dangling pointer, which one crashes your program *today*, and which one might not crash it for a long time?” (Dangling pointers often crash immediately or corrupt visibly; leaks are the slow, silent one — a useful distinction for debugging intuition.)

## 1.7 Simple Contracts: The ADT

**Read alongside:** Sedgewick & Wayne, §1.2, p.64; §1.3, p.120; Karumanchi, Ch. 1, pp.18–19 (PDF); Aho–Hopcroft–Ullman, Ch. 1 (chapter-level).

We can now build a structure and reach it through a pointer. The remaining difficulty is describing one without drowning students in pointer mechanics every time — which is exactly the payoff promised back in Section 1.3’s live question: the data equivalent of “a named, reusable block of logic” is the ADT.

**Definition (Abstract data type (ADT)).** *A description of what we can do with data, without saying how it is stored: a promise about operations [3].*

Sedgewick & Wayne state it as cleanly as anyone: a data type is “a set of values and a set of operations on those values” (p.64) [3]. Karumanchi’s treatment makes the same contract concrete by splitting it into two named parts: an ADT “consists of two parts: 1. Declaration of data[;] 2. Declaration of operations” (p.18, PDF) [2]. The two framings agree exactly — worth pointing out to students as evidence this is a stable idea, not one author’s idiosyncratic phrasing.

**Board: the bag contract, write it first (instance 1).** Write, one line at a time:

1. `insert(x)` — put `x` into the bag.
2. `member(x)` — answer whether `x` is present.
3. `size()` — report how many items are present.

Pause after writing all three and underline: *the contract does not say whether the bag uses an array or a list*. That silence is the feature, not an omission — say this explicitly, since students’ instinct is to treat an unstated detail as a gap to fill in immediately.

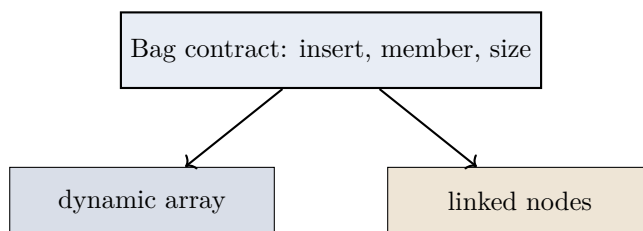


Figure 1.11: One contract, two implementations. Same operations, different internal arrangement; the caller’s code does not change.

**Board: one contract, two implementations (instance 2, interactive classify).** Draw the contract box first, alone, and only add the two implementation boxes below it afterward, connected by “implements” arrows (Figure 1.11) — the build order matters: contract first mirrors the course’s own rule (state the contract before writing a line of the implementation). Then run the deck’s classify drill live, by show of hands:

1. “`member(7)` answers yes or no.” — **contract**: what users can ask.
2. “The values are stored in an array.” — **implementation**: how.
3. “`size()` reports the number of values.” — **contract**: what users can ask.

**Board: distinct instance 3 (design question).** Pose a variant contract — “a queue of integers with `enqueue` and `dequeue`” — and ask which of the two implementation boxes the class



*guesses* is cheaper for `dequeue`. Either answer is pedagogically fine here; the point is that both *could* implement it, which is exactly the abstract claim the bag example just made. Sedgewick & Wayne’s §1.3 (p.120) frames bag/queue/stack as three collections differing only in which object is removed next — worth a one-minute pointer for curious students.

**Board: the course roadmap, ADT by ADT.** List on the board: Linked Lists, Stacks, Queues, Priority Queues, Binary Trees, Hash Tables, Graphs — the commonly used ADTs, in Karumanchi’s phrasing (pp.18–19, PDF) [2]. Point out this is not a random list: it is the Weeks 3–12 work-list, and every one of those weeks opens the same three-beat way this section just did — “What is a Stack?”, then “Stack ADT”, *then* implementation. Today’s bag is the class’s first rehearsal of a pattern the course repeats eight more times.

**Teaching note:** Karumanchi2017 is a strong anchor for the *shape* of the ADT idea and for this roadmap, but it is deliberately not cited anywhere in the stack, heap, or pointer sections above — indexing (Step 0.5) confirmed it has zero coverage there of structured programming or of C’s heap-allocation calls. When answering student questions live: cite Karumanchi2017 for “what is an ADT” and “what’s coming”; cite Wirth2004 for the C memory-management mechanics. Mixing the two up in front of the class is an easy, avoidable slip.

**Teaching note:** The ADT section is where classes mentally check out unless it is *used*. The classify drill is the anchor — it forces the what-vs-how split out of each student rather than letting them nod along. Do not present the ADT as a vocabulary item; present it as a design decision, and preview explicitly that Weeks 3 and 7 implement the *same* stack twice, over an array and then over linked nodes, so the choice becomes real rather than hypothetical. This is deliberately the lightest-weight theoretical point of the lecture, matching the intro-tier difficulty — do not over-formalize it with axioms or algebraic laws; that is a later-course concern.

**Live question:** “we said a function is a named, reusable block of logic, and today we’ve named the data equivalent an ADT. What’s the one-sentence version of the parallel?” (State the contract/interface before writing the implementation — for functions *and* for data, the exact same discipline, one level up.)

## 1.8 Synthesis and Summary

Close by returning to the three prediction boxes from Section 1.2’s opening frame and filling them in together with the class: *where* = the stack (short-lived locals) and the heap (run-time requests); *how* = `malloc/sizeof/calloc/realloc` obtain a heap block and `free` returns it, all reached through a pointer handle; *describe* = an ADT states operations before any implementation. The three boxes are the three of the four opening questions this Act answers with mechanism; question 1 (structuring logic) was answered by the Structured Programming Act itself, and its payoff — “the same discipline, reapplied to data” — is the sentence that ties the whole lecture together.

**Quick reference.** *Stack:* automatic, scope-bound, grows down; each call pushes a frame, each return pops it. *Heap:* run-time sized, lifetime-controlled by you, grows up; `malloc/calloc` obtain, `realloc` resizes, `free` returns, NULL means “nothing.” *Pointer:* stores an address; `->` reads a field; never read through NULL. *Allocation idiom:* `malloc+sizeof` for unspecified content, `calloc` for zero-initialized content, `realloc` to resize (never overwrite the old pointer before checking the result) `→` check NULL `→` initialise every field `→` use `→ free + set NULL`. *Two mistakes:* leak (never freed, silent) and dangling pointer (used after free, often loud); every allocation has exactly one owner. *ADT:* operations stated without storage; a data structure is one implementation of the contract.

Set up Week 2's hook explicitly, tying back to Section 1.2's Sedgewick quote: we can now build two implementations of the same bag, but we have no way to say which is better — only “it felt fast,” which Sedgewick's §1.4 (p.172) already told us is too vague to answer precisely. Week 2 supplies the vocabulary —  $T(n)$ , asymptotic classes, best/worst/average case — to compare implementations without ever running either one.

## References

- [1] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures*. Galgotia Booksource, 2nd edition, 2008.
- [2] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.
- [3] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.
- [4] Niklaus Wirth. *Algorithms and Data Structures*. Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.